Control.Category: https://hackage.haskell.org/package/base/docs/Control-Category.html
Control.Arrow: https://hackage.haskell.org/package/base/docs/Control-Arrow.html

a → b
a × b
x ↦ v

Category

```
-- Control.Category
class Category cat where
    id :: cat a a
    (.) :: cat b c -> cat a b -> cat a c


class Category a => Arrow a where
  {-# MINIMAL arr, (first | (***)) #-}
  arr :: (b -> c) -> a b c
  first :: a b c -> a (b,d) (c,d)
    -- first f    ~   (b,d) -> (f(b),d)
  second :: a b c -> a (d,b) (d,c)
    -- second f   ~   (d,b) -> (d,f(b))

  (***) :: a b c -> a b' c' -> a (b,b') (c,c')
    -- algebraic product

    --  f *** g   ~   (b,b') -> (f(b),g(b'))
  (&&&) :: a b c -> a b c' -> a b (c,c')
    --  f &&& g   ~   c -> (f(c),g(c))

(<<<) :: cat b c -> cat a b -> cat a c
        -- g <<< f   f:a->b  g:b->c
(>>>) :: cat a b -> cat b c -> cat a c
        -- f >>> g
```

Arrow

```
-- Control.Arrow
(^>>) :: Arrow a => (b -> c) -> a c d -> a b d
        -- b -> a (f(b))   f :: b -> c
(>>^) :: Arrow a => a b c -> (c -> d) -> a b d
        -- b -> f (a(b))   f :: c -> d

(<<^) :: Arrow a => a c d -> (b -> c) -> a b d
(^<<) :: Arrow a => (c -> d) -> a b c -> a b d


class Arrow a => ArrowZero a where
  zeroArrow :: a b c

class ArrowZero a => ArrowPlus a where
  (<+>) :: a b c -> a b c -> a b c
```

```
class Arrow a => ArrowChoice a where
             -- Arrow if-else
  {-# MINIMAL (left | (+++)) #-}
  left :: a b c -> a (Either b d) (Either c d)
  right :: a b c -> a (Either d b) (Either d c)

  (+++) :: a b c -> a b' c' -> a (Either b b') (Either c
c')     -- algebraic sum

             --  f +++ g    ~     Left b -> Left (f b)

             --                   Right b' -> Right (g
b)
  (|||) :: a b d -> a c d -> a (Either b c) d
             -- OR

             --  f ||| g     ~    Right b -> f b

             --                   Left c -> g c
class Arrow a => ArrowApply a where
             -- Arrow join
    app :: a (a b c, b) c

class Arrow a => ArrowLoop a where
    loop :: a (b,d) (c,d) -> a b c

newtype Kleisli m a b = Kleisli { runKleisli :: a -> m
b }

newtype ArrowMonad a b = ArrowMonad (a () b)
```

Category
Typeclassopedia: http://wiki.haskell.org/Typeclassopedia

Category
Category from **category theory**. i.e. a set of **objects** with **morphisms**.
A type with
- an **identity** function
- a **composition**

Category laws

```
f.id      == f            Right identity
id.f      == f            Left identity
f.(g.h)   == (f.g).h      Associativity
```

```
---- utils
--
-- composition
(.) :: cat b c -> cat a b -> cat a c
(<<<) :: cat b c -> cat a b -> cat a c
```

```
(>>>) :: cat a b -> cat b c -> cat a c
(<<<) = (.)
(>>>) = flip (.)

-- e.g. (-> r)
-- i.e. functions
(+1)<<<(*2)      -- 2x+1
(+1).(*2)        -- same
(+1)>>>(*2)      -- 2(x+1)
(*2).(+1)        -- same
```

Arrow
Typeclassopedia: http://wiki.haskell.org/Typeclassopedia

Arrow
An arrow represents a **category morphism**.

Laws

```
                      arr id  ==  id
               arr (f >>> g)  ==  arr f >>> arr g
                   x ↦ f(g(x))
                                  where f,g :: (-> r)
               first (arr f)  ==  arr (first f)
                 (x,y) ↦ (f(x),y)
                                  where f::(-> r)
            first (f >>> g)  == first f >>> first g
              (x,y) ↦ (g(f(x)),y)
          first f >>> arr fst  ==  arr fst >>> f
                 (x,y) ↦ (f(x),y)[0] == (x,y) ↦ f(x)
    first f >>> (arr id *** g)  ==  arr (id *** g) >>>
first f            (x,y) ↦ (id(f(x)),g(y)) ==  (x,y) ↦
(f(id(x)),g(y))
                                  where g::(-> r)
 first (first f) >>> arr assoc  ==  arr assoc >>> first
f                 ((x,y),z) ↦ (f(x),(y,z))
                                  where assoc
((x,y),z) = (x,(y,z))
```

Derivations

```
-- minimal: first or ***
first :: Arrow a => a b c -> a (b, d) (c, d)
second :: Arrow a => a b c -> a (d, b) (d, c)
(***) :: Arrow a => a b c -> a b' c' -> a (b, b') (c,
c')
(&&&) :: Arrow a => a b c -> a b c' -> a b (c, c')


swap (x,y) = (y,x)
```

```
-- (***) as first
f *** g = first f >>> arr swap >>> first g >>> arr swap

-- first as (***)
first = (*** id)

-- Derived
second f = arr swap >>> first f >>> arr swap
second = (id ***)
f &&& g = arr split >>> f *** g
  where split x = (x,x)
```

Arrows vs monads
Arrow: https://www.haskell.org/arrows/index.html
Arrow is a **generalization** of monads.
Monadic computation can be represented as a **composition** of **Kleisli functions :: Monad m => a -> m b**
Arrows is a generalization to a **composition** of generalized **arrows :: Arrow arrow => arrow a b**
Hence Arrow is a generalization of Monad to arbitrary arrows.

(<=<) :: Monad m => (b -> m c) -> (a -> m b) -> a -> m c Kleisli composition

(.) :: Arrow arrow => arrow b c -> arrow a b -> arrow a c Arrow composition

Note: Monad (**Kleisli** arrow) ⊂ Arrow
A monad is a special case of Arrow

```
import Control.Arrow hiding (Kleisli(..))
import Control.Category
import Control.Monad
import Prelude hiding ((.))
import Prelude qualified

dot :: (b -> c) -> (a -> b) -> a -> c
dot = (Prelude..)

newtype Kleisli m a b = Kleisli (a -> m b)
        -- Kleisli as arrow

instance (Monad m) => Category (Kleisli m) where
  id = Kleisli return
  Kleisli l . Kleisli k = Kleisli $ k >=> l
        -- Kleisli composition (>=>) as arrow
composition (.)

instance (Monad m) => Arrow (Kleisli m) where
  arr f = Kleisli $ dot return f
  first (Kleisli k) = Kleisli $ \(x, y) -> (,y) <$> k x

instance (Monad m) => ArrowChoice (Kleisli m) where
  left (Kleisli k) = Kleisli $
```

```
    either ((Left <$>) . k) (return . Right)

instance (Monad m) => ArrowApply (Kleisli m) where
        -- Monad join
  app = Kleisli $ \(Kleisli k, x) -> k x
```

Conversely, an Arrow with **ArrowApply** (i.e. monad join) is a monad

```
import Control.Arrow hiding (Kleisli (..))
import Control.Category
import Control.Monad
import Prelude hiding (id, (.))

newtype MyArrow a b c = MyArrow (a b c)

instance (Arrow a) => Functor (MyArrow a b) where
  fmap f (MyArrow k) = MyArrow $ arr f . k

instance (Arrow a) => Applicative (MyArrow a b) where
  pure x = MyArrow $ arr (const x)
  (<*>) :: MyArrow a b (c -> d) -> MyArrow a b c ->
MyArrow a b d
  MyArrow j <*> MyArrow k = MyArrow $ arr (\(x, f) -> f
x) . (k &&& j)

instance (ArrowApply (MyArrow a), Arrow a) => Monad
(MyArrow a b) where           -- join with ArrowApply
  x >>= k = ((x >>> arr k) &&& id) >>> app
```

Basics

```
---- relationship:  i.e. minimal definition
--
```

Arrow utils

```
-- returnA: identity map
returnA :: Arrow a => a b b
returnA = id
returnA 1 = 1
returnA 2 = 2


-- ^ utils: >>>/<<< + arr

-- i.e. same as >>>/<<< + arr in one argument
-- e.g.
--   ^>> = >>> + arr first
--   >>^ = >>> + arr second
(^>>) :: Arrow a => (b -> c) -> a c d -> a b d
(>>^) :: Arrow a => a b c -> (c -> d) -> a b d
```

```
(^>>) f g = arr f >>> g
(>>^) f g = f >>> arr g

-- reverse
(<<^) :: Arrow a => a c d -> (b -> c) -> a b d
(^<<) :: Arrow a => (c -> d) -> a b c -> a b d
(^<<) f g = arr f <<< g
(<<^) f g = f <<< arr g


-- e.g. pure functions (->)
-- All exactly the same since arr=id
(*10) ^>> (+1)        -- 10x+1
(*10) >>^ (+1)        -- same
(*10) >>> (+1)        -- same
(+1) . (*10)          -- same

-- reverse
(*10) ^<< (+1)        -- 10(x+1)
(*10) <<^ (+1)        -- same
(*10) <<< (+1)        -- same
(*10) . (+1)          -- same
```

ArrowChoice

ArrowChoice

Arrows on **algebraic sum** types
i.e. on **Either a b** inputs
4 possibilities:
- **left**: map if left branch
- **right**: map if right branch
- **+++**: map with either branch
- **|||**: map with either branch + merge branches

```
---- Map one branch only
-- i.e.
--  left = map only left
--  right = map only right
left :: ArrowChoice a => a b c -> a (Either b d) (Either
c d)
right :: ArrowChoice a => a b c -> a (Either d b)
(Either d c)
left


---- (+++): algebraic sum of types
(+++):: ArrowChoice a => a b c -> a b' c' -> a (Either b
b') (Either c c')

-- dual of (***)
-- i.e.
--  *** = cartesian product
--  +++ = algebraic sum
(+++):: ArrowChoice a => a b c -> a b' c' -> a (Either b
b') (Either c c')
(***) :: Arrow a => a b c -> a b' c' -> a (b, b') (c,
```

```
c')
(+1) +++ (*10) $ Left 1        -- Left 2
(+1) +++ (*10) $ Right 1       -- Right 10
(+1) *** (*10) $ (1,1)         -- (2,10)


---- (|||): merge
(|||) :: ArrowChoice a => a b d -> a c d -> a (Either b
c) d

-- dual of (&&&)
-- i.e.
--  &&& = split
--  ||| = merge
(|||) :: ArrowChoice a => a b d -> a c d -> a (Either b
c) d
(&&&) :: Arrow a => a b c -> a b c' -> a b (c, c')
(+1) ||| (*10) $ Left 1        -- 1
(+1) ||| (*10) $ Right 1       -- 10
(+1) &&& (*10) $ 1             -- (1,10)
```

ArrowLoop
ArrowLoop

An arrow that allows **recursion**.
i.e. a class of **maps** which allow recursion

Loop laws

```
                loop (arr f)  ==  arr(\b -> fst(fix (\
(c,d) -> f(b,d)))        extension i.e. recursion

                          loop{f}  ==  fst (f (b,d))
 where d = snd (f (b,d))

                          i.e. d = fixed point
        loop (first h >>> f)  ==  h >>> loop f
                          left tightening


                          x ↦ loop{(u,v) ↦ f(h(u)),v)}
(x)

                          x ↦ loop{f}(h(x))
        loop (f >>> first h)  ==  loop f >>> h
                          right tightening

                          x ↦ loop{(u,v) ↦ h(a[0]),a[1])
where a=f(u,v)}(x)

                          x ↦ h(loop{f}(x))
   loop (f >>> arr (id *** k)) == loop (arr (id *** k)
>>> f)                   sliding
```

```
                          loop{(u,v) ↦ (a[0],k(a[1]))
where a=f(u,v)}

                           loop{(u,v) ↦ f(u,k(v))}
                 loop (loop f) == loop (arr uassoc >>> f
>>> arr assoc)         vanishing

                          loop{(u,v) ↦ loop{f}(u,v)}

                            loop{(u,(v,w)) ↦ (a[0],
(a[1]),a[2])) where a=f((u,v),w)}
              second (loop f)  ==  loop (arr assoc >>>
second f >>> arr unassoc)    superposing
                                    assoc ((x,y),z) = (x,
(y,z))                          x ↦ loop{f}(x)[1]

                                x ↦ loop{(u,(v,w)) ↦ (u,
(v,f(w)))}(x)
```

loop
Recursion using a fixed point

loop :: a (b, d) (c, d) -> a b c b = input
c = final result. Usually a function of fixed point d
d = fixed point
d = snd (b, d) = snd (c, d)

Canonical form
fn :: (b, d) -> (c, d)
fn (x, y) = (g x y, f y) x = input
g(x,y) = final result computed from input + fixed point with g :: b -> d -> c
y = f(y) = fixed point for some f :: d -> d

e.g. infinite list

```
x = (1:) x                              -- [1,1...]
x = fix (1:)                            -- same
x = loop (\(_,y) -> (y,(1:) y)) ()    -- same  input=_
 output=[1...]  fixed point y=(1:) y

-- e.g. fixed point + compute
f = loop (\(n,y) -> (take n y,(1:) y))
f 1                                        -- [1]
f 2                                        -- [1,1]
...
```

```
---- loop:
--
--  fix vs loop
fix :: forall d. (d -> d) -> d
loop :: a (b,d) (c,d) -> a b c
```

```
x = fix f :: d                          -- fixed point  g
= f g
x = (f x) :: d                          -- same
x = loop (\(_,y) -> (y, f y)) ()        -- same fixed
point  g = f g  i.e. same as above


-- e.g. infinite list
l = (1:) l                                -- [1,1,...]
l = fix (1:)                              -- Same
l = loop (\(_,x) -> (x,(1:) x) ()         -- Same
f = loop (\(n,x) -> (take n x,(1:) x)
f 1                                       -- [1]
f 2                                       -- [1,1]


-- e.g. factorial
--
factorial n =                                      --
 with recursive expression
  if n <= 1 then n else n * factorial (n-1)
factorialFix = fix (\f n ->                        -- with
fix
  if n <= 1 then n else n * f (n-1))
factorialLoop = loop (let                          -- with
ArrowLoop
  g f n =if n <= 1 then n else n * f (n-1)
  in (\(n, fact) -> (fact n, g fact)))             --
fixed point fact = g fact
factorial 5
factorialLoop 5

-- e.g. fibonacci
fibbo n =                                            --
with recursive expression
  if n <= 1 then n else fibbo (n-1) + fibbo (n-2)
fibboFix = fix (\f n ->                              --
with fix
  if n <= 1 then n else f (n-1) + f (n-2)
fibboLoop = loop (let                                --
with ArrowLoop
  g f n = if n <= 1 then n else f (n-1) + f (n-2)
 in (\(n, fib) -> (fib n, g fib)))                   --
fixed point fib = g fib
fibbo 10
fibboLoop 10
```

==========
======

GHC: Arrow notation

Arrow notation: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/arrows.html

TODO

==========
======
OLD

```
-- arr: lift pure function
arr :: Arrow a => (b -> c) -> a b c
-- e.g.
arr (*1)        -- (*1) i.e. unchanged


---- first,second,***:
--
first :: Arrow a => a b c -> a (b, d) (c, d)
second :: Arrow a => a b c -> a (d, b) (d, c)
first f          -- map first only  i.e. (f(x),y)
second f         -- map second only i.e. (x,f(y))
f *** g          -- map both        i.e. (f(x),g(y))

-- e.g.
first (*10) $ (1,2)        -- (10,2)
second (*10) $ (1,2)       -- (1,20)
(*10) *** (*100) $ (1,2)   -- (10,200)


---- ***,&&&: product
-- i.e. cartesian product of two arrows
--  *** = b × b' → c × c'
--  &&& = b -> c × c'
(***) :: Arrow a => a b c -> a b' c' -> a (b, b') (c,
c')
(&&&) :: Arrow a => a b c -> a b c' -> a b (c, c')
f *** g                -- (f(x),g(y))
f &&& g                -- (f(x),g(x))

-- e.g.
(*10) &&& (*100) $ 1        -- (10,100)
(*10) &&& (*100) $ 2        -- (20,200)


---- relationship:  i.e. minimal definition
--
-- first as axiom: second *** &&& can be derived from
first
swap (x,y) = (y,x)
second = arr swap >>> first >>> arr swap
second = (id ***)
f *** g = first f >>> arr swap >>> first g >>> arr swap
f &&& g = arr (\x->(x,x)) >>> f *** g
```

Arrow utils

```
-- returnA: identity map
returnA :: Arrow a => a b b
returnA = id
returnA 1 = 1
returnA 2 = 2


-- ^ utils: >>>/<<< + arr

-- i.e. same as >>>/<<< + arr in one argument
-- e.g.
--   ^>> = >>> + arr first
--   >>^ = >>> + arr second
(^>>) :: Arrow a => (b -> c) -> a c d -> a b d
(>>^) :: Arrow a => a b c -> (c -> d) -> a b d
(^>>) f g = arr f >>> g
(>>^) f g = f >>> arr g

-- reverse
(<<^) :: Arrow a => a c d -> (b -> c) -> a b d
(^<<) :: Arrow a => (c -> d) -> a b c -> a b d
(^<<) f g = arr f <<< g
(<<^) f g = f <<< arr g


-- e.g. pure functions (->)
-- All exactly the same since arr=id
(*10) ^>> (+1)        -- 10x+1
(*10) >>^ (+1)        -- same
(*10) >>> (+1)        -- same
(+1) . (*10)          -- same

-- reverse
(*10) ^<< (+1)        -- 10(x+1)
(*10) <<^ (+1)        -- same
(*10) <<< (+1)        -- same
(*10) . (+1)          -- same
```